

Model Tuning, Ensemble Learning & Unsupervised Learning

A simple, plain-language guide to these notes.

1. Model Tuning

The first version of a model you build is almost never the best one. Model tuning means adjusting a model's settings so it works better.

These settings are called **hyperparameters**. They are things you choose before training, like how deep a decision tree can grow, how many neighbours KNN should look at, or how big the learning rate should be.

If you pick random hyperparameters, the model might underfit (too simple, performs badly everywhere) or overfit (memorises the training data but fails on new data). The goal of tuning is to find the hyperparameter values that make the model perform well on **new, unseen data** — not just the data it was trained on.

2. Cross Validation

Before tuning, we need a fair way to test a model. Normally we split data into 80% training and 20% testing. But that one 20% slice might happen to be easy or hard by chance, giving a misleading score.

Cross validation fixes this by testing more fairly. In **k-fold cross validation**, we split the data into k equal parts (folds). We train on k-1 parts and test on the remaining 1 part — and we repeat this k times, each time using a different part as the test set. At the end we average all the accuracy scores. For example with k = 5, we get 5 accuracy scores, and we average them to get one reliable score for the model.

Example: accuracy scores of 82%, 81%, 84%, 86%, 85% across 5 folds → average = 83.6%. That average is a much more trustworthy score than a single train/test split.

3. Hyperparameter Tuning Methods

Once we can measure a model fairly, we can try different hyperparameter values and see which combination scores best. There are three common ways to search for the best combination:

- **Manual Search** — you try a few combinations yourself, one at a time, and compare the results by hand. Slow, and easy to miss the best combination.
- **Grid Search CV** — you list out all the values you want to try for each hyperparameter, and the computer tries *every possible combination* automatically, using cross validation to score each one. For example, if KNN has 6 values for 'n_neighbors', 2 values for 'weights', and 2 values for 'metric', grid search tries $6 \times 2 \times 2 = 24$ combinations and picks the best.

- **Randomized Search CV** — instead of trying every combination, it randomly samples a fixed number of combinations. This is much faster when there are a huge number of possible combinations (e.g. tuning XGBoost with learning rate, number of estimators, and max depth could mean $10 \times 20 \times 10 = 2000$ combinations — too many to try all of, so we randomly sample a smaller number, like 30).

Common hyperparameters to tune: Ridge/Lasso Regression → alpha (learning rate); KNN → number of neighbours; Decision Tree → max depth, min sample split, max features; SVM → kernel type.

4. Ensemble Learning

The idea behind ensemble learning is simple: 'we trust the crowd more than one person.' Instead of relying on a single model's prediction, we combine the predictions of **several models** to get a better, more reliable final answer.

For example, say 4 different models (Logistic Regression, SVM, Naive Bayes, KNN) each predict whether someone is a 'hacker' based on typing speed and internet usage. If 3 models say yes and 1 says no, the ensemble's final answer is yes — this is called **majority voting**. For regression problems (predicting a number instead of a category), instead of voting, the ensemble takes the **average** of all the models' predictions.

There are three common types of ensemble learning: **Bagging**, **Boosting**, and **Stacking**.

4.1 Bagging (Bootstrap Aggregation)

Bagging trains many copies of the **same kind** of model on different random samples of the data, then combines their answers by voting (or averaging, for regression).

How it works: if you have 2000 rows of data, you randomly pick smaller samples (say 1000 rows each, with repetition allowed) to create several different training sets (D1, D2, D3, D4...). You train one model on each sample (M1, M2, M3, M4...). Each model gives its own prediction, and the final answer is decided by majority vote.

Random Forest is the most well-known bagging technique — it trains many decision trees (for example 100 trees), each on a different random sample of the data and a different random subset of features, then combines all their votes into one final prediction.

Bagging is mainly used to fix **overfitting** (low bias, high variance — the model does great on training data but poorly on test data). By averaging many overfitted models trained on different samples, the combination becomes more generalised.

4.2 Boosting

Boosting trains models **one after another**, not all at once. Each new model focuses extra attention on the mistakes the previous model made, gradually turning several 'weak learners' (models that are only slightly better than guessing) into one 'strong learner'.

The final prediction is a weighted combination of all the models: $f = \alpha_1(m_1) + \alpha_2(m_2) + \alpha_3(m_3) + \dots + \alpha_n(m_n)$, where each α is a weight showing how much to trust that model.

Boosting is mainly used to fix **underfitting** (high bias, high variance — the model is too simple and performs badly overall). By chaining models that keep correcting each other's errors, the end result becomes both accurate and generalised.

*Popular boosting algorithms: **AdaBoost**, **Gradient Boosting**, and **XGBoost** (Extreme Gradient Boosting — a faster, more powerful version of gradient boosting, very popular in real-world competitions and jobs).*

4.3 Stacking

Stacking trains several **different types** of models (called 'base models' — e.g. SVM, Decision Tree, Logistic Regression) on the same data. Instead of just voting, their predictions are fed as input into one more model, called the **meta model** (often KNN or Logistic Regression), which learns the best way to combine the base models' predictions into a final answer.

So the flow is: data → base models (SVM, Decision Tree, Logistic Regression) each make a prediction → those predictions become the input features for the meta model → the meta model gives the final prediction.

4.4 Bias & Variance, in Simple Terms

- **High bias** = the model is too simple and makes systematic mistakes even on training data (underfitting).
- **High variance** = the model is too sensitive to the training data and changes a lot if the data changes slightly, doing well on training data but badly on new data (overfitting).
- The goal is always **low bias and low variance** — a model that's accurate and also generalises well to new data. Bagging mainly reduces variance, boosting mainly reduces bias.

5. Unsupervised Learning

So far, every model discussed needs **labelled** data — data where the correct answer is already known (this is called **Supervised Learning**). **Unsupervised Learning** is different: the data has **no labels** at all. There's no 'right answer' given — the model has to find patterns and structure in the data all by itself.

Supervised Learning	Unsupervised Learning
Data has inputs and labels (the output is known)	No labels — only input data
The model learns to predict the label	The model learns patterns in the data
Examples: Classification, Regression	Examples: Clustering, Dimensionality Reduction, Anomaly Detection

Why use unsupervised learning? To explore data we don't fully understand yet, find hidden groups or patterns, reduce how complicated the data is, spot unusual/outlier data points, or clean up data before feeding it into a supervised model (e.g. using PCA before a classifier).

Real-world uses: grouping customers by spending habits, detecting fraud, recommending similar products, compressing images, and finding patient groups in medical data.

6. Clustering: K-Means

Clustering means grouping similar data points together, without being told the groups in advance.

K-Means is the most common clustering algorithm. Steps:

- Decide how many clusters you want (this number is called k). Example: $k = 4$.
- Randomly place k 'centroids' (starting center points) in the data.
- Assign every data point to its nearest centroid (measured using straight-line, i.e. Euclidean, distance).
- Recalculate each centroid as the average position of all points assigned to it, and move it there.
- Repeat the assign-and-recalculate steps until the centroids stop moving — the clusters are now final.

How do you choose the right value of k ? Using the **Elbow Method**. We calculate a score called **WCSS** (Within-Cluster Sum of Squares — basically, how far points are from their own cluster's center, squared and summed up) for different values of k , and plot them. As k increases, WCSS always drops, but at some point the drop becomes much smaller — that bend in the curve looks like an elbow, and the k at that bend is usually the best choice.

K-Means has some limitations: you must choose k yourself in advance, outliers can drag a centroid away from where it should be, and it struggles with clusters that aren't roughly circular in shape.

7. Clustering: DBSCAN

DBSCAN is a different kind of clustering algorithm that groups points based on how **densely packed** they are, instead of using a center point like K-Means does. This means it can find oddly-shaped clusters and doesn't need you to decide the number of clusters in advance.

It works using a distance called **epsilon** — points that are within epsilon distance of each other, and packed closely enough, are grouped into the same cluster. Points that don't fit into any dense group are treated as outliers/noise.

In short: K-Means = centroid-based algorithm, DBSCAN = density-based algorithm.

8. The Curse of Dimensionality

When a dataset has a very large number of features (columns) — sometimes hundreds — several problems start to appear:

- It becomes very difficult for models to find real patterns in the data.
- Distance-based algorithms like KNN, K-Means and DBSCAN start to fail, because distance calculations stop being meaningful when there are too many dimensions.
- The data takes up a lot more storage space.
- Models take much longer to train.

This whole set of problems is called the **Curse of Dimensionality**. The fix is **Dimensionality Reduction** — reducing the number of features while keeping as much important information as possible (e.g. going from 150 features down to 50).

9. Two Ways to Reduce Dimensions

- **Feature Selection** — simply pick a smaller subset of the existing features and drop the rest, without changing them. Example: out of features x, y, z, you decide to keep only x and y.
- **Feature Extraction** — combine existing features into brand-new features that capture the same information more compactly. Example: 'engine width' and 'engine height' can be combined into a new single feature called 'engine size'. Some detail is always lost in this process, but usually not enough to hurt predictions much.

10. PCA (Principal Component Analysis)

PCA is the most common feature extraction technique. Instead of keeping the original axes (features) as they are, PCA finds new axes — called **principal components** (PC1, PC2, ...) — that point in the direction where the data **varies the most**.

PC1 is drawn along the direction of maximum variance (spread) in the data, and PC2 is drawn perpendicular to it, capturing the next most variance, and so on. By keeping only the top few principal components (e.g. just PC1 instead of all original features), we shrink the number of dimensions while keeping most of the useful information — for example, going from 3D down to 2D, or from 50 dimensions down to 10.

In short, PCA always looks for the line (or plane) along which the data spreads out the most, because that's the direction that carries the most information.